

Lab 03 – Collections, Generic Lists, Searching

Title: Collections and Data Management in Memory

Objectives

Now that I had my employee class defined, I needed a way to store and manage collections of employees. The goal was to understand how to use generic collections (specifically `List<T>`) and implement basic CRUD operations. I also learned about the importance of separating concerns by creating a dedicated project for data storage.

Implementation

IStocareData Interface

```
using SalariiModel;
using System.Collections.Generic;

namespace SalariiStocare
{
    public interface IStocareData
    {
        void AddAngajat(Angajat angajat);
        List<Angajat> GetAngajati();
        List<Angajat> CautaDupaNume(string nume);
        List<Angajat> CautaDupaFunctie(string functie);
        bool ModificaAngajat(int id, Angajat angajatNou);
        bool StergeAngajat(int id);
    }
}
```

```
}
```

In-Memory Storage Implementation

```
using SalariiModel;

using System;

using System.Collections.Generic;

using System.Linq;

namespace SalariiStocare
{
    public class AdministrareAngajatiMemorie : IstocareData
    {
        private List<Angajat> _angajati = new List<Angajat>();
        private int _nextId = 1;

        public void AddAngajat(Angajat angajat)
        {
            angajat.Id = _nextId++;
            _angajati.Add(angajat);
        }

        public List<Angajat> GetAngajati()
        {
            return _angajati;
        }

        public List<Angajat> CautaDupaNume(string nume)
        {
            return _angajati.Where(a =>
                a.Nume.Contains(nume, StringComparison.OrdinalIgnoreCase) ||
```

```

        a.Prenume.Contains(nume, StringComparison.OrdinalIgnoreCase)
    ).ToList();
}

```

```

public List<Angajat> CautaDupaFunctie(string functie)
{
    return _angajati.Where(a =>
        a.Functie.Contains(functie, StringComparison.OrdinalIgnoreCase)
    ).ToList();
}

```

```

public bool ModificaAngajat(int id, Angajat angajatNou)
{
    var existing = _angajati.FirstOrDefault(a => a.Id == id);
    if (existing == null) return false;

    existing.Nume = angajatNou.Nume;
    existing.Prenume = angajatNou.Prenume;
    existing.Functie = angajatNou.Functie;
    existing.Departament = angajatNou.Departament;
    existing.TipSalariu = angajatNou.TipSalariu;
    existing.SalariuBrut = angajatNou.SalariuBrut;
    existing.TarifOra = angajatNou.TarifOra;
    existing.OreLucrate = angajatNou.OreLucrate;
    existing.Bonusuri = angajatNou.Bonusuri;
    existing.DataAngajarii = angajatNou.DataAngajarii;
    return true;
}

```

```

public bool StergeAngajat(int id)

```

```

{
    var existing = _angajati.FirstOrDefault(a => a.Id == id);

    if (existing == null) return false;

    return _angajati.Remove(existing);
}
}
}

```

Why I Used This Approach

Interface (IStocareData): The interface defines a contract that any storage implementation must follow. This promotes loose coupling – the UI doesn't need to know how data is stored, just that it can call these methods. This makes it easy to switch storage mechanisms later (e.g., from memory to file to database).

List<T>: I chose List<T> because it's the most versatile generic collection in .NET. It provides O(1) access by index, supports LINQ queries, and has built-in methods like Add, Remove, and Find.

Auto-Increment ID: The _nextId field ensures each employee gets a unique ID. This is crucial for identifying records in a database-like manner.

Case-Insensitive Search: Using StringComparison.OrdinalIgnoreCase makes the search user-friendly – searching for "pop" finds both "Pop" and "Popescu".

LINQ for Searching: Where with a lambda expression provides a clean, readable way to filter collections. The ToList() method materializes the results into a new list, preserving the original collection.

Testing & Results

I tested the system with a console application:

```
var admin = new AdministrareAngajatiMemorie();
```

```
// Add employees
```

```
admin.AddAngajat(new Angajat { Nume = "Popescu", Prenume = "Ion", Functie = "Programator" });
```

```
admin.AddAngajat(new Angajat { Nume = "Ionescu", Prenume = "Maria", Functie = "Manager"
});

admin.AddAngajat(new Angajat { Nume = "Popa", Prenume = "Andrei", Functie = "Analist" });

// Test search

var rezultate = admin.CautaDupaNume("pop");

// Returns: Popescu Ion and Popa Andrei (case-insensitive)

// Test modify

admin.ModificaAngajat(1, new Angajat { Nume = "Popescu", Prenume = "Ion", Functie =
"Senior Programator" });

// Test delete

admin.StergeAngajat(2); // Removes Ionescu Maria
```

Reflection

Using `List<T>` with LINQ made the code very concise and readable. I could have written the search manually with loops, but LINQ expresses the intent much more clearly. The separation of concerns between Model, Storage, and UI is already paying off – the code is more organized and easier to maintain. The interface pattern will be crucial when we implement file-based persistence in a later lab.